GYMBOT

Un chatbot para reservar en tu gimnasio

Proyecto de desarrollo · Arquitectura y lógica

FastAPI · Groq LLM · SQLite · Gradio



Made with GAMMA

El problema de acceso

CONSULTAR Y RESERVAR DEBERÍA SER TAN SIMPLE COMO CHATEAR

Las llamadas y los formularios rígidos generan fricción para socios y administración.

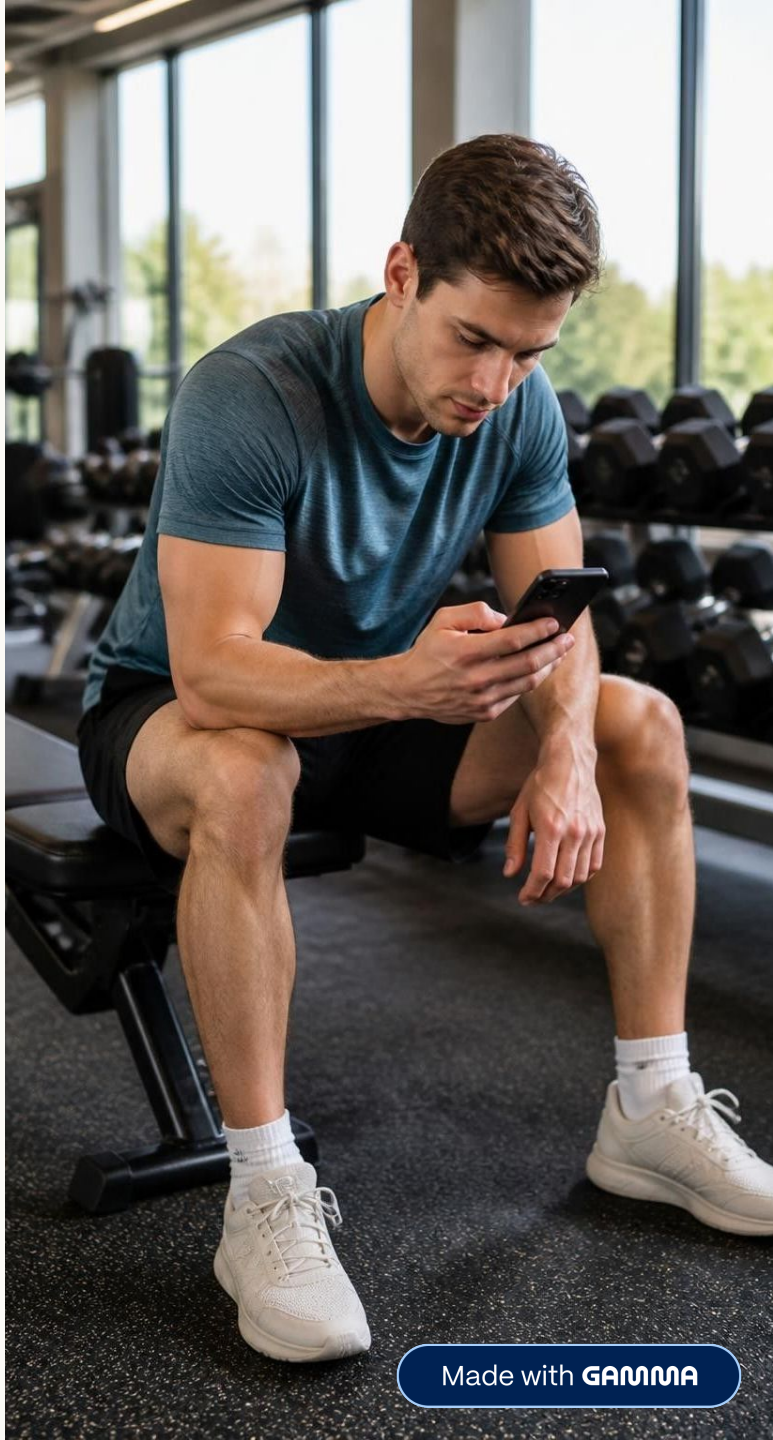
1. **Planes, precios y horarios en español natural**

   Preguntas cotidianas sin navegar menús complejos ni esperar respuestas en recepción.

---

2. **Reservas inmediatas sin llamadas**

   Agendamiento directo con confirmación clara en pantalla y trazabilidad completa.



Made with GAMMA

QUÉ PUEDE HACER GYMBOT

Tres ejes operativos resueltos desde el mismo canal de conversación.

1



Información completa

Consulta planes, precios actualizados, catálogo de clases y normas del gimnasio.

2



Disponibilidad real

Verifica cupos por fecha, servicio y franja horaria antes de cualquier solicitud.

3



Ciclo de reservas

Crear, consultar, modificar y cancelar turnos empleando un código alfanumérico único.

Made with GAMMA

Pila tecnológica · 5 componentes

CINCO TECNOLOGÍAS, CADA UNA CON UN PAPEL

Separación nítida entre comprensión lingüística, orquestación, validación y almacenamiento.



Groq LLM

Interpreta la intención y extrae entidades con mínima latencia de inferencia.



Python

Controla la lógica de negocio, reglas de transición y estados de confirmación.



FastAPI

Expone endpoints REST estructurados con validación estricta de esquemas.



SQLite

Persiste turnos de forma relacional, transaccional y resistente a reinicios.



Gradio

Provee la interfaz web interactiva con botones contextuales y streaming de chat.

Made with GAMMA

Flujo de datos end-to-end

DEL MENSAJE DEL USUARIO A LA BASE DE DATOS

1

Gradio

Entrada UI

El socio redacta su solicitud en el chat

→

2

Memoria

Contexto

Recupera historial y estado de sesión

→

3

Groq LLM

Extracción

Identifica intención y parámetros clave

→

4

Lógica Python

Control

Valida acción y exige confirmación

→

5

Persistencia

FastAPI + SQLite

Verifica cupo y guarda en reservas.db

Flujo síncrono validado en cada capa

Made with GAMMA

Modular

CADA ARCHIVO TIENE UNA SOLA RESPONSABILIDAD

Arquitectura limpia con límites claros entre presentación, servicio y datos.

Sin acoplamiento directo entre el modelo LLM y la base de datos.

`main.py`	API y validación
`almacen_sqlite.py`	persistencia y disponibilidad
`app_gradio.py`	interfaz y botones
`chatbot_memoria.py` y `consulta_llm.py`	conversación y Groq
`servicios.json` y `negocio.json`	catálogo y reglas

Made with GAMMA

Principio de diseño seguro

LA IA INTERPRETA, PERO PYTHON DECIDE

- Crear, modificar o cancelar exige confirmar en la interfaz
- Las escrituras comprueban disponibilidad antes de guardar
- Una reserva cancelada conserva su registro y cambia de estado



Made with GAMMA

REGLAS DE RESERVA QUE EVITAN CHOQUES

Condiciones rígidas implementadas en el backend para asegurar la integridad de la agenda.

1

Clases grupales

60 minutos.

Capacidad topada en 10 personas por bloque horario sin sobrecupos.

2

Valoración física

60 minutos.

Atención individual disponible únicamente de lunes a viernes en agenda hábil.

3

Fechas válidas

Solo futuro.

El backend rechaza cualquier solicitud con fecha u hora anterior al momento actual.

4

Control de cruces

HTTP 409 Conflict.

Si el horario solicitado no tiene capacidad, la API rechaza el intento limpiamente.

Made with GAMMA



Arquitectura de persistencia

LAS RESERVAS SOBREVIVEN AL REINICIO

1 **Reservas en SQLite**

`reservas.db` conserva las reservas.

2 **Chat en Gradio**

El chat vive en la sesión de Gradio.

Reiniciar la conversación no borra reservas.

Made with GAMMA

Despliegue local rápido

PUESTA EN MARCHA EN CUATRO PASOS

1. Clonar el repositorio y crear el entorno virtual con `python -m venv venv`.
2. Instalar paquetes de `requirements.txt` y configurar `GROQ_API_KEY` en `.env`.
3. Ejecutar migraciones iniciales y levantar FastAPI en el puerto `8011`.
4. Ejecutar `app_gradio.py` y abrir el chat web interactivo en el puerto `7860`.

Listo para demostración en vivo y pruebas de concurrencia local.

4 PASOS

Del repositorio al chat funcional en menos de 5 minutos

Made with GAMMA